

UD4. Despliegue de aplicaciones con Docker

Licencia:

Creative Commons Atribución-NoComercial-CompartirIgual (CC BY-NC-SA 4.0)

1.- Despliegue de aplicaciones



Una aplicación, en términos de informática, es un programa o conjunto de programas diseñados para ayudar a los usuarios a realizar una serie de tareas específicas. Estas pueden variar desde procesamiento de textos, navegación web, hasta funciones más complejas como la edición de video o la gestión de bases de datos. Cada aplicación está construida para operar en un sistema operativo particular y aprovechar sus recursos para ejecutar las tareas para las que fue diseñada.

En un servidor se pueden desplegar diversos tipos de aplicaciones, dependiendo de la infraestructura y las necesidades del proyecto. A continuación, indicaremos algunos de los **tipos de aplicaciones más comunes que pueden ser desplegadas en un servidor**:

1. Aplicaciones Web:

Estas son aplicaciones accesibles a través de un navegador web, generalmente compuestas por un frontend y un backend.

Ejemplos: Sitios web, plataformas de comercio electrónico, blogs, portales web corporativos.

Tecnologías comunes: HTML, CSS, JavaScript, frameworks como React, Angular, Vue.js, y backend en Node.js, Python (Django, Flask), Ruby on Rails, PHP (Laravel, WordPress), ASP.NET.

2. Aplicaciones Backend/Servicios API

Estas son aplicaciones o servicios que proporcionan lógica empresarial o procesamiento de datos. Las APIs son un ejemplo clave de aplicaciones backend.

Ejemplos: APIs REST, APIs GraphQL, microservicios.

Tecnologías comunes: Node.js, Python, Ruby, Java, Go, C#, Spring Boot, Express.js, Flask.

3. Aplicaciones de Bases de Datos

Son sistemas que almacenan, gestionan y consultan datos. Estas aplicaciones requieren servidores dedicados o contenedores que gestionen bases de datos.

Ejemplos: MySQL, PostgreSQL, MongoDB, Redis, Cassandra, MariaDB.

Tipos de bases de datos: Relacionales, NoSQL, In-memory, Grafos.

4. Aplicaciones de Mensajería/Colas

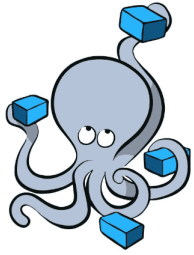
Aplicaciones que gestionan el envío, la recepción y el almacenamiento temporal de mensajes entre diferentes servicios.

Ejemplos: RabbitMQ, Apache Kafka, ActiveMQ.

Uso: Procesamiento de mensajes en aplicaciones distribuidas, microservicios, colas de trabajos.

Desplegar una aplicación en un servidor no es tan simple como podría parecer. Uno de los grandes desafíos es gestionar las dependencias y las librerías que la aplicación necesita para funcionar correctamente. Dependencias son otros programas o módulos que tu aplicación necesita para ejecutarse. Estas deben estar presentes y correctamente configuradas en el servidor para que la aplicación funcione. Además, diferentes aplicaciones a menudo requieren versiones específicas de estas librerías y dependencias, lo que puede llevar a conflictos si múltiples aplicaciones se ejecutan en el mismo entorno de servidor. La

2.- La solución: Docker compose



docker
Compose

Docker Compose es una herramienta que facilita la definición y gestión de aplicaciones multi-contenedor en Docker. Permite al usuario escribir un archivo YAML para configurar los servicios, redes y volúmenes necesarios para una aplicación. Esto simplifica el proceso de configuración al definir todas las necesidades de una aplicación en un solo lugar. Docker Compose se utiliza principalmente en los entornos de desarrollo, pruebas y producción, proporcionando una manera consistente y reproducible de desplegar aplicaciones.

Ayuda a simplificar y automatizar el despliegue de aplicaciones en múltiples contenedores, lo que es especialmente útil en entornos de servidor donde la gestión de dependencias y librerías puede ser compleja. Al utilizar Docker Compose, cada contenedor puede ser configurado para incluir sus propias librerías y dependencias, eliminando el riesgo de conflictos entre aplicaciones. Esto es particularmente valioso en entornos donde se despliegan múltiples aplicaciones que pueden tener requisitos contradictorios.

Además, Docker Compose facilita la escalabilidad de las aplicaciones permitiendo definir claramente cómo deben interactuar y escalar los contenedores dentro de una aplicación. También mejora la consistencia entre los entornos de desarrollo y producción, ya que los desarrolladores pueden definir y compartir configuraciones específicas para la aplicación, asegurando que se comportará de manera similar en todos los entornos. Esto reduce los errores comunes que pueden surgir debido a diferencias en la configuración del entorno, haciendo que el proceso de despliegue sea más predecible y seguro.

Ejemplos donde Docker Compose tiene sentido:

Con Docker Compose se pueden desplegar diversos tipos de aplicaciones, especialmente aquellas que requieren varios servicios para funcionar de manera integrada. Aquí algunos ejemplos de los tipos de aplicaciones que se pueden desplegar:

- **Aplicaciones web tradicionales:** Las aplicaciones que siguen la arquitectura cliente-servidor, como sitios web o aplicaciones web basadas en tecnologías como Node.js, Python (Django, Flask), Ruby on Rails o PHP, pueden desplegarse fácilmente con Docker Compose. Estos despliegues a menudo requieren servicios adicionales como bases de datos o servidores de caché.
- **Aplicaciones microservicios:** Docker Compose es ideal para manejar aplicaciones que están construidas en torno a microservicios, donde cada servicio puede estar encapsulado en un contenedor. Por ejemplo, en una aplicación de microservicios, podrías tener un contenedor para la API, otro para la base de datos, otro para el servidor de autenticación, etc.
- **Aplicaciones con bases de datos:** Las aplicaciones que dependen de bases de datos como MySQL, PostgreSQL, MongoDB, Redis o Elasticsearch pueden ser fácilmente configuradas con Docker Compose. Este archivo YAML permite definir la configuración de la base de datos y cómo interactúa con otros contenedores.
- **Aplicaciones de análisis de datos:** Herramientas como Jupyter Notebooks o aplicaciones de procesamiento de datos con Apache Spark, Hadoop, o bases de datos orientadas a análisis como Apache Cassandra, pueden desplegarse utilizando Docker Compose para manejar servicios distribuidos.
- **Aplicaciones de mensajería y colas:** Aplicaciones que dependen de sistemas de mensajería como RabbitMQ, Kafka, o ActiveMQ pueden ser desplegadas junto con otras partes de la aplicación (como productores y consumidores de mensajes) usando Docker Compose.
- **Aplicaciones de procesamiento de imágenes o multimedia:** Aplicaciones que requieren procesamiento de imágenes, videos o archivos multimedia, como servicios de compresión o conversión de formatos,

también pueden ser desplegadas con Docker Compose, junto con los servicios adicionales necesarios (almacenamiento, procesamiento paralelo, etc.).

- Aplicaciones de CI/CD (Integración continua/Despliegue continuo): Herramientas como Jenkins, GitLab CI, o Travis CI pueden ejecutarse en contenedores junto con otras herramientas auxiliares como agentes de construcción, servicios de prueba automatizados o servidores de despliegue.
- Aplicaciones basadas en APIs: Servicios backend orientados a proveer APIs RESTful o GraphQL pueden ser configurados y desplegados con Docker Compose. Además, puedes incluir servicios relacionados como balanceadores de carga, sistemas de autenticación o motores de búsqueda.
- Aplicaciones con servicios en la nube simulados localmente: Docker Compose puede ser utilizado para simular entornos de la nube localmente, permitiendo ejecutar servicios como bases de datos en memoria, almacenamiento distribuido, o servidores de caché en contenedores que replican el comportamiento de servicios en la nube como AWS, Azure o GCP.

2.1.- Instalación, configuración y comandos

Anteriormente, hemos visto los **comandos básicos** para la gestión de contenedores, aunque existen muchos otros. La configuración de forma manual de los contenedores puede ser una tarea laboriosa, repetitiva y en la cual podemos olvidar algunos pasos al realizarlas varias veces.

La primera idea que viene a la mente es la de la creación de un guión que ejecute los comandos de forma automática, si bien no es una mala práctica, tiene varios inconvenientes como la **portabilidad** o la **falta de estandarización**.

Docker posee una herramienta que permite la gestión automatizada de la configuración de los contenedores llamada **Docker Compose**. Esta herramienta facilita la configuración de contenedores usando un fichero en formato **YAML** .

Un ejemplo típico es la de una aplicación web, una base de datos y un servidor de correo para la gestión de usuarios/contraseñas, en el que cada uno de ellos se ejecuta en un contenedor diferente, pero que forman parte de la misma aplicación, gracias a **Compose**. Poner en marcha los contenedores se realiza ejecutando únicamente un comando.

Instalación de Docker Compose

En **Windows** y **MAC** viene incluido con **Docker Desktop** por lo que no es necesario realizar ninguna acción, y en **Linux** puedes comprobar la versión que tienes:

```
docker compose version
```

y en caso de no tenerlo instalado ejecutar los siguientes comandos:

```
sudo apt update
sudo apt install docker-compose-plugin
```

Formato fichero docker-compose

El fichero **docker-compose.yml** especifica los servicios a levantar y, sus características, se encuentran escritas en formato **YAML**. La especificación del fichero se puede encontrar en:

<https://github.com/compose-spec/compose-spec/blob/main/spec.md>

El elemento principal es el servicio, el cual se define dentro de la propiedad **services** del fichero. Estos servicios son una definición abstracta de un recurso informático dentro de una aplicación y se encuentran respaldados por contenedores que se ejecutan en función de los requisitos de la aplicación. En Docker Compose, los nombres de los servicios actúan como nombres de host dentro de la red interna de Docker. Cada servicio dispone para su configuración de decenas de propiedades. Algunas de ellas son:

- **image**: indica una imagen base a usar
- **build**: especifica la configuración de compilación para crear una imagen de contenedor a partir del código fuente
 - **context** define una ruta a un directorio que contiene un Dockerfile o una URL a un repositorio Git

- **dockerfile_inline** : especifica el nombre de un fichero Dockerfile alternativo para construir la imagen o una configuración con sintaxis Dockerfile en el propio compose

■ **deploy**: Permite configurar el despliegue del servicio, posee propiedades como:

- **labels** para añadir meta-información al servicio
- **mode** que indica cómo se realiza la replicación en caso de ser necesario
- **replicas** para especificar el número de contenedores que han de ejecutarse al mismo tiempo
- **resources** son requisitos normalmente hardware para el servicio, como el número de CPUS o la memoria. Se pueden indicar los límites y establecer reservas
- **constraints** para fijar requisitos de ejecución de la plataforma como que el dispositivo de almacenamiento sea SSD
- **restart_policy** configurar cómo se reinician los contenedores, indicando si existe un retraso (*delay*), un número máximo de intentos (*max_attempts*)

■ **container_name**: Fija el nombre del contenedor.

■ **dns**: define servidores de nombres personalizados a usar en el contenedor (fichero /etc/resolv.conf)

■ **domainname**: declara un nombre personalizado a usar en el contenedor (fichero /etc/hosts)

■ **depends_on**: orden de inicio y arranque en función de otros servicios como, por ejemplo, que el servicio de base de datos se ha de iniciar antes que el servidor web

■ **restart**: cuando uno de los contenedores termina, establece la política de reinicio:

- no: La política de reinicio predeterminada. No reinicia el contenedor bajo ninguna circunstancia.
- always: La política siempre reinicia el contenedor hasta que se elimine.
- on-failure[:max-retries]: La política reinicia el contenedor si el código de salida indica un error. Opcionalmente, se puede limitar el número de intentos de reinicio que realiza el daemon de Docker.
- unless-stopped: La política reinicia el contenedor sin importar el código de salida, pero deja de reiniciarlo cuando el servicio se detiene o se elimina.

■ **logging**: permite configurar el **log del contenedor**, por ejemplo definir el sistema **syslog** e indicar la **url** en el que se almacenan los **logs**

■ **network**: Se utiliza para establecer la comunicación entre los servicios. Se especifican con un nombre. A su vez posee diferentes propiedades como la IPv4 e IPv6 o la prioridad.

■ **ports**: Puertos en los que el servicio se encuentra escuchando. Posee diferentes formatos (rango de puertos, **ip** o **tcp/udp**).

■ **read_only**. El sistema de ficheros del contenedor es de solo lectura.

■ **volumes**: Define los volúmenes a usar por el contenedor y en qué lugar se almacenan en el anfitrión. Es posible compartir volúmenes en varios servicios. Algunas de las propiedades son:

- **type**: Tipo de montaje, **bind**, **tmpfs** o **npipe**.
- **source**: Directorio del *host* anfitrión.
- **target**: Ruta en el que se montará en el contenedor.
- **read_only**: Indica que el volumen es de solo lectura.

Comando docker-compose

Una vez creado el fichero, se ejecuta el comando **docker compose** para la gestión de esa configuración, con este comando es posible administrar los contenedores, las imágenes, los servicios en ejecución, así como obtener información de la ejecución del sistema. Las diferentes opciones del **CLI** se pueden encontrar en: <https://docs.docker.com/reference/cli/docker/compose/> las más utilizadas son:

- **docker compose up**: A partir de *docker-compose.yml* crea los contenedores y los inicia.
- **docker compose down**: apaga los servicios levantados.
- **docker compose ps**: Equivalente al comando **ps** del sistema operativo.
- **docker compose exec**: Similar al comando **exe** de **Docker**.
- **docker compose log**: Permite ver los **logs** de los servicios.
- **docker composer rm**: Borra un servicio (contenedor) previamente parado.

En resumen

Docker Compose es una herramienta que simplifica la gestión de aplicaciones conformadas por múltiples contenedores, también llamadas stack de aplicación. Su configuración se basa en un archivo YAML, permitiendo un despliegue ordenado y estructurado de servicios. Los pasos para definir un stack con Docker Compose son:

- Crear los archivos Dockerfile correspondientes a cada servicio
- Definir los servicios en el fichero `docker-compose.yml`
- Ejecutar el comando `docker compose up` para inicializar la aplicación

Dentro del fichero YAML, se pueden especificar diferentes elementos clave:

- **Services**: especifica la configuración de cada contenedor en el stack, y se parece a los parámetros que recibiría si lo ejecutamos con docker run.
- **Networks**: define las redes personalizadas que usarán los servicios del mismo modo que lo haríamos con el comando `docker network create`
- **Volumes**: permite gestionar el almacenamiento persistente del mismo modo que lo haríamos con el comando `docker volume create`
- **Configs y Secrets**: Facilita la gestión de configuraciones y credenciales de forma segura.

Gracias a Docker Compose, es posible manejar entornos de manera aislada, realizar actualizaciones parciales de servicios y **escalar** aplicaciones fácilmente ejecutando múltiples instancias de un mismo servicio.

3.- Red

Al igual que cuando trabajamos con máquinas virtuales, si queremos conectar servicios o contenedores distintos, utilizaremos las posibilidades que Docker nos ofrece, también llamadas "drivers de red". **Docker Compose por defecto crea una red bridge** donde estarán conectados todos los contenedores definidos. Este comportamiento puede ser redefinido en la configuración, creando nuevas redes o conectando los contenedores a redes preexistentes. Otros elementos de la comunicación, como direcciones IP, puertos expuestos, hostnames y DNS se pueden configurar como parte de los servicios individuales.

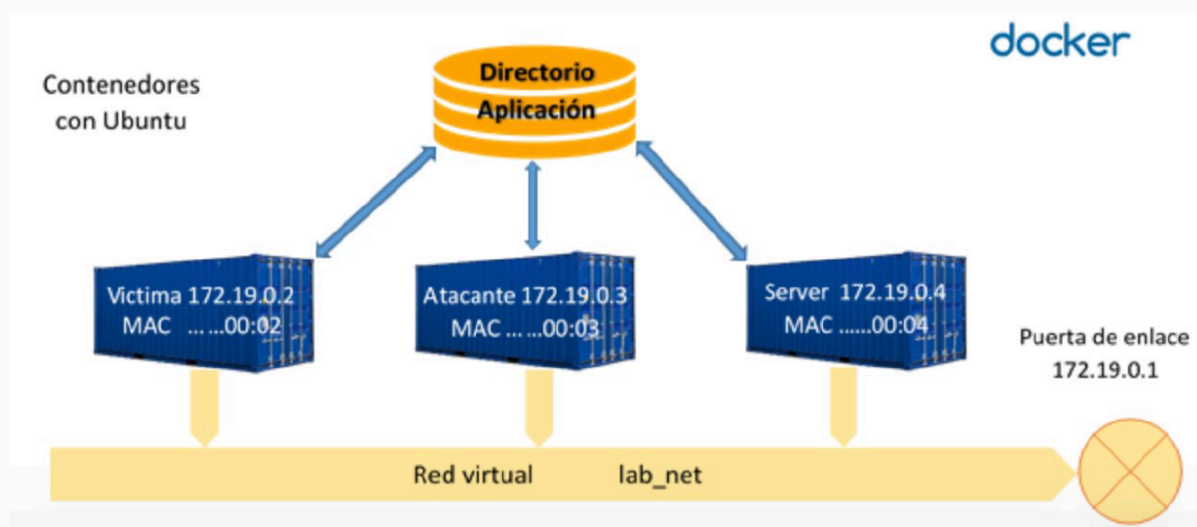
Opciones y tipos

- Opciones:

```
docker network (create | ls | inspect | rm)
```

A continuación se introducen los distintos tipos de driver de red que nos ofrece docker:

Driver	Descripción
bridge	Opción por defecto: todos los contenedores están en la misma red, separada del host
host	Quita el aislamiento de red entre contenedor y el host Docker
none	Aislado completo tanto del host como del resto de contenedores
overlay	Las redes overlay sirven para conectar varios Docker daemons entre sí (sería como una agrupación lógica de Dockers instalados en diferentes host). La red overlay multihost no se instala por defecto con Docker
macvlan	La manera de asignar una dirección MAC a un contenedor, se usa cuando se necesita que el contenedor pueda acceder directamente a la red física



El uso de contenedores conectados a la red por defecto no está recomendado en entornos de producción. se podrían enlazar contenedores con el bridge por defecto con el flag `--link`. La opción recomendable es definir una red bridge y crear los contenedores conectados a dicha red:

- Por ejemplo:

```
docker network create delariojared
```

- Se crean los contenedores conectados a dicha red:

```
docker run -d --name servidor_mysql --network delariojared -v mariadb_data:/var/lib/mysql \
-e MYSQL_DATABASE=wordpress -e MYSQL_USER=wp_user -e MYSQL_PASSWORD=org \
-e MYSQL_ROOT_PASSWORD=org mariadb
docker run -d --name servidor --network delariojared delarioja/probandoweb:v1
```

En entornos Windows, hay controladores equivalentes como NAT (similar a Bridge en Linux) y Transparent (equiparable a Macvlan).

Las redes en Docker pueden administrarse mediante el comando `docker network`, además de permitir la personalización de configuraciones como DNS, reglas de firewall (iptables) y soporte para IPv6. Por ejemplo, el comando recomendado para comprobar cómo conectaron los contenedores anteriores, sería:

```
docker network inspect delariojared
```

Dentro de los host Linux, Docker modifica las reglas de iptables (cortafuegos de Linux) para garantizar el aislamiento en la red segmentando el tráfico de paquetes correctamente. Lo recomendado es no manipular estas reglas para no afectar el funcionamiento de las políticas aunque en equipos expuestos/públicos podría hacerse en la cadena DOCKER-USER con el fin de tener mayor control. Consultar la documentación oficial en dichos casos.

DNS

Los contenedores utilizan la configuración de DNS del host, tal como se establece en los ficheros `/etc/hosts` y `/etc/resolv.conf`.

Sin embargo, esta configuración puede cambiarse con las opciones `"dns"`, `"dns-opts"` y `"dns-search"` del fichero de configuración de Docker Daemon:

- **dns**: define una lista de IP de servidores DNS.
- **dns-opts**: define una lista de pares clave/valor de opciones de DNS.
- **dns-search**: establece un nombre de dominio donde buscar los nombres de hostname que no sean un nombre de dominio completo (FQDN).

Igualmente, al **crear un contenedor se pueden incluir los flags** `--dns`, `--dns-opts`, `--dns-search` y `--hostname`, este último para cambiar el nombre de dominio del contenedor.



Para saber más

<https://www.youtube.com/embed/BNHNMoSJz4g>

Descripción

(#exe-accordion-1-0)

En este vídeo explicativo se hace una breve introducción al networking en Docker: tipos de drivers, ejemplos, etc.

<https://www.youtube.com/embed/J2als0JX5iY>

Descripción

(#exe-accordion-2-0)

En este vídeo más completo se presentan los fundamentos del networking en Docker: objetivos, infraestructura base: Container Network Model (CNM), ejemplo de overlay con Swarm, etc.

4.- Laboratorio. Despliegues

En este laboratorio pondremos en práctica lo más importante viendo algunos ejemplos de despliegues con Docker compose.

4.1.- Wordpress

En este vídeo abordaremos el despliegue del gestor de contenidos WordPress mediante docker-compose y más concretamente con el fichero, que deberás renombrar para lanzarlo por defecto: [wordpress-docker-compose.yml](https://www.youtube.com/embed/r4ko8kzZ4Bg)

<https://www.youtube.com/embed/r4ko8kzZ4Bg>

Requisitos a cumplir:

- Despliega WordPress tal que el servidor web esté escuchando en el puerto 4282.
- Despliega PhpMyAdmin de modo que esté a la escucha en el puerto 4283.
- La versión del sistema gestor de base de datos será MySQL 5.7
- Las credenciales de MySQL serán: usuario->miusuario, password->mipassword, nombre de la base de datos->wordpress.
- Hay que referenciar en el servicio de WordPress al host de MySQL, que no puede ser localhost o 127.0.0.1, sino una referencia al servicio de MySQL.
- Allí donde se precise, el usuario root de MySQL será 1234.

También puedes usar este otro más actual pero sin el PhpMyAdmin: [wordpress-docker-compose2.yml](#)

4.2.- Moodle

En este vídeo abordaremos el despliegue del LMS Moodle mediante docker-compose y más concretamente con el fichero, que deberás renombrar para lanzarlo por defecto: [moodle-docker-compose.yml](#)

<https://www.youtube.com/embed/Oc8842PG8bk>

Requisitos a cumplir:

- Despliega Moodle tal que el servidor web esté escuchando en el puerto 8001.
- La versión de Moodle a instalar será la 4.3.
- La versión del sistema gestor de base de datos será bitnami/mariadb:11.2
- Las credenciales de MariaDB serán: usuario->bn_moodle, password->debe poder ser vacío, nombre de la base datos->bitnami_moodle.
- Genera volúmenes que referencien en Moodle estos directorios: /bitnami/moodle y /bitnami/moodledata.

4.3.- Aplicación Java

En este vídeo abordaremos el despliegue de una aplicación Java mediante [java-docker-compose.yml](#) y [MiClase.java](#)

<https://www.youtube.com/embed/mlBgVgHVFh8>

Requisitos a cumplir:

- Debe hacer uso de la imagen del JDK openjdk en su versión 11.
- Hará uso de la clase Java MiClase.java (facilitada más abajo).
- La tarea debe compilar la clase MiClase, obteniendo MiClase.class
- La tarea debe ejecutar el archivo compilado.
- La tarea debe empaquetar el archivo compilar en un archivo de extensión .jar
- Debe mapearse mediante un volumen el directorio /app del contenedor respecto al directorio ./app del host.

4.3.- Node.js

En este vídeo abordaremos el despliegue de Node.js mediante [docker-compose](#) junto con [Dockerfile](#) (y [server.js](#) por si quieres reproducirlo)

<https://www.youtube.com/embed/at1VaA7I1Pg>

Requisitos a cumplir:

- Generar una imagen con Node.js versión 14
- Desplegar un servicio web con Node.js escuchando en el puerto 8080 y para cualquier dirección IP
- Verificar que en el navegador recibimos una respuesta del servidor en <http://ip-del-servidor:8080>

5.- Práctica "Dockerizando ando"



Ejercicios obligatorios

Vamos a poner a prueba los conceptos estudiados en la teoría de la unidad didáctica, que cubre los conceptos sobre contenedores y docker relacionados con Docker compose.



Requisitos Iniciales:

- Visiona los vídeos del laboratorio de la Unidad 4.
- Realizar el examen online de la unidad, y consulta nuevamente las dudas que te surjan.
- Acceso a una terminal de Ubuntu con privilegios de administrador/root o -una vez instalado Docker Engine- pertenecer al grupo "docker".
- No olvides elaborar el documento explicativo a partir de la plantilla y ver previamente el siguiente vídeo con el enunciado completo de la tarea:

<https://www.youtube.com/embed/7c4NHd610k0>

Para que entiendas la parte del .env necesitarás saber: ¿cómo funcionan las variables de entorno en Docker Compose? (#exe-accordion-3-0)

■ Se cargan automáticamente

Si hay un archivo `.env` en el mismo directorio que `docker-compose.yml`, Docker Compose lo lee sin necesidad de especificarlo. Después leerá la sección `environment` en caso de que exista en el `.yml`

■ Prioridad de variables

- Variables definidas en la terminal (`export VAR=valor`) sobrescriben las del `.env`.
- Variables en `docker-compose.yml` sobrescriben las del `.env`.

- Las variables en `.env` tienen menor prioridad, pero permiten una configuración predeterminada.

Objetivo:

En caso de que no puedas ver el vídeo anterior o no te quede claro qué dockerizar, éste sería el objetivo:

Queremos instalar Prestashop. Para ello haz uso de docker compose. Prestashop es una aplicación web de e-commerce, deberás realizar los siguientes apartados.

Apartados 1 y 2 - Instalación de MySQL y phpMyAdmin

- Configura docker-compose.yml para poder desplegar un servicio de MySQL junto con phpMyAdmin
- Tras ejecutar `docker compose` tendrás que disponer de un phpMyAdmin en el puerto 8091 del host y un MySQL conectado al mismo
- Adjunta la captura que demuestre que puedes acceder a phpMyAdmin y crear un usuario "prestashop" con privilegios totales sobre una base de datos con el mismo nombre
- Baja el entorno y comprueba que los servicios están parados

Apartado 3 - Instalación de PrestaShop.

- Añade al mismo docker-compose.yml anterior el servicio que permita desplegar PrestaShop
- Tras ejecutar `docker compose` tendrás que poder acceder a PrestaShop en el puerto del host que hayas decidido
- Adjunta la captura que demuestre que puedes acceder a PrestaShop y a la base de datos que creaste en el apartado anterior

Apartado 4 - Visualización de la base de datos.

- Visualiza las tablas en phpMyAdmin que componen TODA la instalación de Prestashop
- Tendrás que configurar en docker-compose las credenciales correctamente para alcanzar el punto anterior



Ejercicios opcionales: puede entregarse uno, varios, todos o ninguno

1. Realiza el tipo test de la unidad 4 hasta que saques más de un 8 sobre 10 en la calificación. Manda una captura de pantalla insertada en el documento/plantilla cuando lo tengas.
2. Este ejercicio se propone para que puedas testar cómo funciona el aislamiento de redes con Docker. Para ello, puedes seguir los siguientes pasos:
 - Crea una red bridge llamada `network1`
 - Crea una red bridge llamada `network2`
 - Comprueba que las redes se han creado correctamente
 - Ejecuta dos contenedores conectados a `network1`
 - Ejecuta un contenedor conectado a `network2`
 - Ejecuta un intérprete interactivo en uno de los contenedores de `network1`
 - Comprueba la conectividad con el otro contenedor de `network1`
 - Comprueba que no es posible conectarse al contenedor de `network2`

💡 **Sugerencia:** Apóyate en la ayuda en línea de `docker network` para determinar las opciones necesarias. Puedes usar la imagen `alpine` para los contenedores dado que incorpora el comando `ping`

3. Sigue los ejemplos de red propuestos en los vídeos del apartado 3 sobre Networking en Docker ajustándolos a tu manera en tu máquina virtual (por ejemplo si la interfaz de la Rpi en el vídeo de Pelado sale `eth0` habrá que poner en nuestra máquina `enp0s3` en su lugar).

5.1. Evaluación de la tarea

Criterios de evaluación implicados

¿Cómo valoro tu tarea?

Rúbrica de la tarea	
Ejercicio 1:	2 p.
Ejercicio 2:	2 p.
Ejercicio 3:	2,5 p.
Ejercicio 4:	2,5 puntos
Bibliografía web y Valoración personal	0,5 + 0,5 puntos
Ejercicios Opcionales y Retos:	hasta + 0.25 puntos extra cada uno
Ejercicios de Ampliación, Comentarios detallados	hasta + 1 punto extra cada uno

5.2. Para saber más

Glosario

Las palabras que debes entender en esta unidad relativa a Docker son:

- Un **STACK** de aplicaciones también llamado stack de soluciones, es una lista de todos los servicios tecnológicos utilizados para construir y ejecutar una sola aplicación. En una aplicación Web, por ejemplo, un stack podría ser el software necesario para implementar el frontend (React.js, Webpack, Sass), el backend (Node.js, Express), la base de datos (MongoDB) y el servidor Web (Nginx).
- **YAML**: YAML es un formato de serialización de datos legible por humanos y utilizado en múltiples lenguajes de programación. Se caracteriza porque la jerarquía entre elementos se realiza indentando las líneas de menor jerarquía. Un fichero YAML es más simple de entender que otras alternativas de serialización, como XML o JSON.
- Una **dirección MAC** (Media Access Control) es un identificador único de 48 bits asignado a la tarjeta de red de un dispositivo para su comunicación en redes locales (LAN). Se expresa en formato hexadecimal, como `00:1A:2B:3C:4D:5E` y permite la identificación y control del tráfico en una red.

Seguridad en Docker

Al contenerizar aplicaciones con Docker, hay que prestar especial cuidado a la seguridad. Los puntos más vulnerables son:

- El sistema operativo del host.
- La instancia del Docker Daemon.
- El software de cada imagen.
- Los procesos de los contenedores.
- El almacenamiento de la información secreta.

Para proteger el host, lo más importante es mantener actualizado el sistema, aplicando actualizaciones críticas y apoyándose en módulos de seguridad como AppArmor, Grsecurity y SELinux.

Gestión de logs

Los servicios contenerizados suelen generar logs que permiten auditar el funcionamiento de las aplicaciones y revisar los eventos de interés. Docker proporciona múltiples drivers para conectar la salida de logs del contenedor con agregadores de logs locales o remotos, como CloudWatch de AWS.

Integración Continua con Docker

La integración continua (CI) y la entrega continua (CD) son procesos de automatización de etapas repetitivas del desarrollo. Los pasos clave incluyen:

- Construcción de la imagen.
- Ejecución de las pruebas.
- Etiquetado de la imagen.
- Subida de la imagen al registro.

Contenedores en producción

Para mejorar el rendimiento y eficiencia en producción, es fundamental:

- Optimizar las imágenes para reducir su tamaño y agilizar la transferencia.
- Configurar correctamente el uso de memoria RAM y CPU.
- Asegurar que las herramientas de orquestación estén optimizadas.

Orquestación de contenedores

Kubernetes es una plataforma portable y extensible para administrar cargas de trabajo, contenedores y servicios. Entre sus ventajas destacan:

- Autoescalado horizontal.
- Balanceo de carga.
- Despliegues y rollbacks inteligentes.
- Gestión de configuración y credenciales de acceso.

Las cargas de trabajo en Kubernetes incluyen ReplicaSet, StatefulSet, DaemonSet, Job y CronJob.

<https://www.youtube.com/embed/videoseries?list=PL3HUVK90ReCLGgtcg7-MK5i2sPNHoe499>

Lista de vídeos del módulo

En esta lista iré dejando los vídeos explicativos que iré haciendo sobre Docker: instalación, ejemplos, etc.

